

Jupyter Notebook Execution Report

Name: Amanda Quintino dos Santos
Project Title: BU OMDS
Date: June 28, 2026

Cell 1: ■ Markdown

Support Vector Machine (SVM) Exploration of predict_conversion_df

Cell 2: ■ Code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score

# 1. Load data
predict_conversion_df = pd.read_csv("digital_marketing_campaign_dataset.csv")

# 2. Separate features and target
X = predict_conversion_df.drop(columns=['CustomerID', 'Conversion'])
y = predict_conversion_df['Conversion']

# 3. Identify column types
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
cat_cols = X.select_dtypes(include=['object']).columns

# 4. One-Hot Encode Categorical features
X = pd.get_dummies(X, columns=cat_cols, drop_first=True)

# 5. Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 6. Scale Numerical features (fit on train, transform on both)
```

```

scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

# 7. Initialize and train SVM with RBF kernel
print("Training SVM (RBF Kernel)...")
svm_rbf = SVC(kernel='rbf', random_state=42)
svm_rbf.fit(X_train, y_train)

# 8. Predict and Evaluate
y_pred = svm_rbf.predict(X_test)

print("\n--- SVC (RBF Kernel) Results ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred))

```

Output:

```

Training SVM (RBF Kernel)...

--- SVC (RBF Kernel) Results ---
Accuracy: 0.8919
Classification Report:

```

	precision	recall	f1-score	support
0	0.68	0.21	0.32	194
1	0.90	0.99	0.94	1406
accuracy			0.89	1600
macro avg	0.79	0.60	0.63	1600
weighted avg	0.87	0.89	0.87	1600

Cell 3: ■ Markdown

Support Vector Machine Exploration of google_ads_df

Cell 4: ■ Code

```

# 1. Load data
google_ads_df = pd.read_csv("GoogleAds_DataAnalytics_Sales_Uncleaned.csv")

```

```

# 2. Clean numeric strings
for col in ['Cost', 'Sale_Amount']:
    if google_ads_df[col].dtype == 'object':
        google_ads_df[col] = google_ads_df[col].str.replace('$', '',
            regex=False).astype(float)

# 3. Handle missing data instead of dropping
# Fill features
google_ads_df.fillna(0, inplace=True)

# 4. Separate features and target
median_conv = google_ads_df['Conversions'].median()
X2 = google_ads_df.drop(columns=['Ad_ID', 'Ad_Date', 'Keyword', 'Conversions',
    'Conversion Rate'])
y2 = (google_ads_df['Conversions'] > median_conv).astype(int)

# 5. One-Hot Encode Categorical features
cat_cols2 = X2.select_dtypes(include=['object']).columns
X2 = pd.get_dummies(X2, columns=cat_cols2, drop_first=True)

# 6. Train/Test Split
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2,
    random_state=42)

# 7. Scale Numerical features
num_cols2 = X2_train.select_dtypes(include=['int64', 'float64']).columns
scaler2 = StandardScaler()
X2_train[num_cols2] = scaler2.fit_transform(X2_train[num_cols2])
X2_test[num_cols2] = scaler2.transform(X2_test[num_cols2])

# 8. Train the SVM with RBF kernel
print("Training SVM (RBF Kernel) for google_ads_df...")
svm_rbf2 = SVC(kernel='rbf', random_state=42)
svm_rbf2.fit(X2_train, y2_train)

# 9. Evaluate
y2_pred = svm_rbf2.predict(X2_test)
print("\n--- SVC (RBF Kernel) Results: google_ads_df ---")
print(f"Accuracy: {accuracy_score(y2_test, y2_pred):.4f}")

```

```
print("Classification Report:\n", classification_report(y2_test, y2_pred))
```

Output:

```
Training SVM (RBF Kernel) for google_ads_df...
```

```
--- SVC (RBF Kernel) Results: google_ads_df ---
```

```
Accuracy: 0.5173
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.54	0.55	0.54	271
1	0.50	0.49	0.49	249
accuracy			0.52	520
macro avg	0.52	0.52	0.52	520
weighted avg	0.52	0.52	0.52	520

```
[STDERR]
```

```
&lt;string&gt;;7: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
```

```
You are setting values through chained assignment. Currently this works in certain cases, but whe
```

```
A typical example is when you are setting values in a column of a DataFrame, like:
```

```
df["col"][row_indexer] = value
```

```
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html
```

```
&lt;string&gt;;7: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
```

```
You are setting values through chained assignment. Currently this works in certain cases, but whe
```

```
A typical example is when you are setting values in a column of a DataFrame, like:
```

```
df["col"][row_indexer] = value
```

```
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html
```

Cell 5: ■ Markdown

Support Vector Machine Exploration: marketing_products_df

Cell 6: ■ Code

```
# 1. Load data
marketing_products_df = pd.read_csv("marketing_and_product_performance.csv")

# 2. Define the target
median_conv3 = marketing_products_df['Conversions'].median()
X3 = marketing_products_df.drop(columns=['Campaign_ID', 'Product_ID',
    'Customer_ID', 'Flash_Sale_ID', 'Bundle_ID', 'Conversions'])
y3 = (marketing_products_df['Conversions'] > median_conv3).astype(int)

# 3. One-Hot Encode Categorical features
cat_cols3 = X3.select_dtypes(include=['object']).columns
X3 = pd.get_dummies(X3, columns=cat_cols3, drop_first=True)

# 4. Train/Test Split
X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size=0.2,
    random_state=42)

# 5. Scale Numerical features
num_cols3 = X3_train.select_dtypes(include=['int64', 'float64']).columns
scaler3 = StandardScaler()
X3_train[num_cols3] = scaler3.fit_transform(X3_train[num_cols3])
X3_test[num_cols3] = scaler3.transform(X3_test[num_cols3])

# 6. Train the SVM with RBF kernel
print("Training SVM (RBF Kernel) for marketing_products_df...")
svm_rbf3 = SVC(kernel='rbf', random_state=42)
svm_rbf3.fit(X3_train, y3_train)

# 7. Evaluate
y3_pred = svm_rbf3.predict(X3_test)
print("\n--- SVC (RBF Kernel) Results: marketing_products_df ---")
print(f"Accuracy: {accuracy_score(y3_test, y3_pred):.4f}")
print("Classification Report:\n", classification_report(y3_test, y3_pred))
```

Output:

```
Training SVM (RBF Kernel) for marketing_products_df...

--- SVC (RBF Kernel) Results: marketing_products_df ---
Accuracy: 0.5095
```

Classification Report:

	precision	recall	f1-score	support
0	0.53	0.50	0.51	1033
1	0.49	0.52	0.51	967
accuracy			0.51	2000
macro avg	0.51	0.51	0.51	2000
weighted avg	0.51	0.51	0.51	2000